



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Laurea Triennale in Ingegneria Informatica

**Programmazione C++ Moderna:
i costrutti del nuovo standard C++26**

Relatore:

Prof: Marco Cococcioni

Candidato:

Sebastiano Pala

ANNO ACCADEMICO 2024/2025

Indice

1	Introduzione	4
1.1	I paradigmi della programmazione	4
1.2	Obiettivo della tesina	4
2	Compile-Time Programming	5
2.1	Template Metaprogramming	5
2.2	Type Traits	6
2.2.1	SFINAE	7
2.3	Constexpr	8
2.3.1	Constexpr negli anni	8
2.4	Metaprogramming nel C++26	11
3	Gestione della Memoria	14
3.1	Gestione manuale delle risorse	14
3.2	Rule of Three	15
3.3	Il principio RAI	15
3.4	Move Semantics	16
3.4.1	Perfect Forwarding	17
3.5	Rule of Five	18
3.6	Smart Pointers	19
3.6.1	Unique Pointers	20
3.6.2	Shared Pointers	20
3.6.3	Weak Pointers	21
3.7	Nuove Proposte	23
3.8	Scope Guards	24
3.9	Gestione Dinamica Efficiente con std::hive	25
4	Programmazione Asincrona	27
4.1	Multithreading nel passato	27
4.2	Supporto nativo del multithreading	27
4.3	Strumenti del multithreading	28
4.4	Parallel STL and std::execution	32
4.4.1	JThread	33
4.5	Le Coroutine	35

5	Conclusioni	40
5.1	Sviluppi futuri	40

Capitolo 1

Introduzione

Il C++ è uno dei linguaggi più importanti nel panorama della programmazione. Pubblicato ufficialmente nel 1985 come estensione del linguaggio C, si è evoluto nel corso degli anni attraverso una serie di standard che hanno esteso le sue funzionalità, restando però retro-compatibile, richiedendo dunque un delicato equilibrio tra introduzione di astrazioni potenti e moderne e la preservazione dell'efficienza e del controllo a basso livello che lo definiscono.

1.1 I paradigmi della programmazione

L'evoluzione del C++ moderno può essere interpretata come uno sforzo verso il rendere sicuri ed espressivi i paradigmi che i programmatori avevano già individuato e implementato, spesso con notevole complessità, utilizzando i costrutti già disponibili nel linguaggio. Il principio RAII (Resource Acquisition Is Initialization), basato sulla gestione sicura delle risorse, è stato alla base della creazione degli smart pointer. La metaprogrammazione a tempo di compilazione, basata su tecniche come lo SFINAE (Substitution Failure Is Not An Error), è stata normalizzata e resa accessibile da costrutti come `constexpr`, e dai `type traits`. Molto importante è anche la spinta verso la possibilità di eseguire algoritmi complessi a tempo di compilazione, in modo da scaricare una parte del carico di lavoro al compilatore, migliorando di conseguenza l'efficienza del codice runtime.

1.2 Obiettivo della tesina

Questa tesina si concentra sull'analisi dei costrutti che saranno disponibili nel futuro standard C++26, partendo dalla loro introduzione nel linguaggio fino a raggiungere la versione finale, analizzando tutti gli aggiornamenti e migliorie ricevute attraverso i vari standard intermedi, fino ad arrivare alle nuove proposte più recenti destinate a far parte del nuovo standard C++26. L'obiettivo non è fornire un semplice elenco descrittivo delle feature, ma valutare l'impatto concreto che questi costrutti hanno sulla scrittura di software robusto ed efficiente.

Capitolo 2

Compile-Time Programming

Questo capitolo analizza l'evoluzione delle tecniche di programmazione a tempo di compilazione in C++. L'obiettivo è quello di sfruttare il compilatore allo scopo di:

- Ottimizzare il runtime: i calcoli, la generazione di codice specializzato e la risoluzione di decisioni vengono spostati dalla fase di esecuzione a quella di compilazione.
- Aumentare la sicurezza e l'espressività: utilizzare il compilatore per imporre vincoli sui tipi di dati e sulla correttezza del codice permette di catturare errori logici e di tipo in fase di compilazione, rendendo i programmi più robusti, oltre a ridurre la necessità di controlli runtime.

2.1 Template Metaprogramming

Il Template Metaprogramming (TMP) consiste nell'utilizzare i template per eseguire calcoli a tempo di compilazione. Sfrutta il meccanismo di istanziamento dei template, spesso in modo ricorsivo. E' una tecnica potente ma complessa, molto difficile da utilizzare e da debuggare, in particolare dovuta al fatto che non è una feature progettata, ma è nata quasi per caso dalla scoperta che il sistema dei template è Turing-completo. Di conseguenza la sintassi è controintuitiva, e gli errori sono notoriamente difficili da interpretare, in quanto spesso consistono in centinaia di righe di output per un singolo errore. Inoltre, l'uso pesante di template ricorsivi può portare a tempi di compilazione significativamente lunghi.

```
1 //Calcolo il fattoriale con TMP
2 template <unsigned n>
3 struct Factorial {
4     enum {value = n * Factorial<n-1>::value};
5 };
6
7 //Specializzazione del template per il
8 //caso base della ricorsione
```

```
9  template <>
10 struct Factorial<0> {
11     enum { value = 1 };
12 };
13
14 //Esempio di utilizzo:
15     const unsigned fact6 = Factorial<6>::value; // 720 a tempo di
        compilazione
```

L'esempio seguente utilizza la tecnica standard dell'enum, ampiamente utilizzata nel C++98/03 per la metaprogrammazione con i template, per definire costanti a tempo di compilazione in modo portabile ed efficiente.

2.2 Type Traits

I type traits sono una parte fondamentale della standard library del C++, e permettono di fare metaprogrammazione sui tipi. Introdotti nel C++11 e inclusi dall'header `<type_traits>`, forniscono un'interfaccia che permette di manipolare e interrogare le proprietà dei tipi, tutto a tempo di compilazione. Sono estremamente utili nella programmazione generica: all'interno di un template, il tipo `T` è sconosciuto, ma grazie ai type traits possiamo adattare il comportamento del nostro codice in base alle proprietà del tipo stesso. I type traits possono essere di due tipi:

- Type predicates: rispondono a domande sui tipi, fornendo come risultato un booleano.

```
1  #include <type_traits>
2
3  std::is_integral<int>::value // true
4  std::is_integral<int*>::value // false
5  std::is_pointer<char*>::value // true
6  std::is_const<const int>::value // true
7
8  //type traits di confronto
9  std::is_same<T,U> //true se T e U sono dello stesso tipo
10 std::is_base_of<Base,Derived> // verifica se Base è una
    classe base di Derived (o lo stesso tipo)
11 std::is_convertible<From, To> // verifica se il tipo From può
    essere implicitamente convertito in To
12
13 //in C++17 e successivi, si può usare _v al posto di ::value
14 std::is_integral_v<int> // true
```

- Type transformations: permettono di costruire un nuovo tipo, a partire da un altro.

```

1 #include <type_traits>
2
3 std::remove_const<const char*>::type // ottengo char*
4 std::make_signed<unsigned int>::type // ottengo int
5 std::decay<T>::type // simula la trasformazione che subisce
    un tipo quando si passa per valore ad una funzione,
    Esempio: array -> puntatore
6
7 //dal C++14:
8 std::remove_const_t<const int> // evito di scrivere ::type

```

2.2.1 SFINAE

Queste informazioni possono essere utilizzate per controllare la compilazione stessa, ad esempio abilitando o disabilitando selettivamente funzioni template. Per ottenere questo risultato, è necessario il costrutto `std::enable_if` che, combinato con i type predicates, permette di pilotare il meccanismo di SFINAE. SFINAE significa "Substitution Failure Is Not An Error". Questo vuol dire che, quando il compilatore tenta di sostituire parametri template durante la risoluzione di un overload, se l'operazione non ha successo a causa di un tipo o di un'espressione non valida, non viene generato un errore ma semplicemente viene scartata quella particolare versione del template dall'insieme dei candidati possibili.

SFINAE in azione

Supponiamo di avere due funzioni omonime, come faccio a far sì che il compilatore scelga quella corretta?

```

1 #include <iostream>
2 #include <type_traits>
3
4 // caso puntatori
5 template <typename T>
6 std::enable_if_t<std::is_pointer_v<T>, void>
7 print(T ptr) { //static cast per il caso T = char* (evito di
    stampare la stringa)
8     std::cout << "INDIRIZZO: " << static_cast<const void*>(ptr)
    << std::endl;
9 }
10
11 // caso numeri
12 template <typename T>
13 std::enable_if_t<std::is_integral_v<T>, void>
14 print(T value) {
15     std::cout << "NUMERO: " << value << std::endl;
16 }
17

```

```
18 int intero = 12;
19 int* pointer = &intero;
20
21 print(intero); //chiama la versione per interi
22 print(pointer); //chiama la versione per puntatori
23 //print(4.12); //ERRORE DI COMPILAZIONE: non viene trovata
    nessuna funzione corrispondente
```

Il compilatore, quando prova a inserire il tipo `int` nel primo template, invece di dare errore, semplicemente scarta la funzione, utilizzando la seconda.

2.3 Constexpr

Introdotta con C++11, la parola chiave `constexpr` ha rivoluzionato la metaprogrammazione, spostando il focus dai template (TMP) a una sintassi basata su funzioni e variabili. Dichiarare una funzione come `constexpr` significa che, se tutti i suoi argomenti sono espressioni costanti, il compilatore la valuterà a tempo di compilazione. Se questa condizione non è soddisfatta, verrà chiamata normalmente a runtime. Rispetto al TMP, `constexpr` offre una sintassi più naturale e intuitiva, ma inizialmente presentava forti limitazioni: non era permesso utilizzare cicli (`for`, `while`), variabili mutabili o dichiarazioni multiple, e veniva accettato solo l'operatore condizionale ternario per le decisioni.

Confronto con TMP

```
1 //calcolo fattoriale
2 constexpr unsigned factorial(unsigned n) {
3     return n <= 1 ? 1 : n * factorial(n - 1);
4 }
5
6 // Utilizzo:
7 constexpr unsigned fact6 = factorial(6); // 720 a tempo di
    compilazione
```

2.3.1 Constexpr negli anni

`Constexpr` era fin dalla prima apparizione uno strumento molto potente, ma negli anni le limitazioni sono state rilassate, permettendo di utilizzare `constexpr` in maniera ancora più efficace.

- C++14: Lo standard introdusse la possibilità di contenere cicli, variabili locali modificabili, e salti condizionati, permettendo di scrivere funzioni `constexpr` con una sintassi più simile al codice runtime.

- C++17: Arriva if constexpr. La metaprogrammazione venne rivoluzionata ulteriormente, permettendo di scrivere codice condizionale che viene valutato a tempo di compilazione. Vengono resi obsoleti molti degli accorgimenti complessi basati su SFINAE per la selezione tra varie implementazioni di una funzione, permettendo di usare un semplice if la cui condizione è basata di solito su type traits.

```

1 #include <iostream>
2 #include <type_traits>
3
4 template <typename T>
5 void print(T input){
6     if constexpr(std::is_pointer_v<T>){
7         std::cout << "INDIRIZZO: " << static_cast<const void
*>(input) << std::endl;
8     }else if constexpr(std::is_integral_v<T>){
9         std::cout << "NUMERO: " << input << std::endl;
10    }
11 }

```

Il compilatore, basandosi sul tipo T, genera una versione della funzione print che contiene solo la sezione di codice pertinente.

- C++20: Arriva il supporto per l'utilizzo della memoria dinamica e strutture dati complesse in contesti constexpr, come std::vector e std::string.

```

1 #include <vector>
2 #include <iterator>
3
4 constexpr int get_pos(int t) {
5     std::vector<int> vec = {50, 21, 300, 10000, 2, 2362};
6     vec.push_back(t);
7     sort(vec.begin(), vec.end());
8
9     auto it = std::find(vec.begin(), vec.end(), t);
10
11     return std::distance(vec.begin(), it)+1;
12 }
13 constexpr int pos = get_pos(27); //Output: 3

```

Il controllo sull'esecuzione a tempo di compilazione viene raffinato con l'introduzione di:

- constexpr: garantisce che una funzione venga sempre valutata a compile-time, generando un errore in caso non sia possibile.

```
1 //Funzione esclusivamente compile-time
2 constexpr int double_it(int n) { return n * 2; }
3 constexpr auto x = double_it(21);
4 // auto y = double_it(n); //errore se n non è constexpr
```

- `constexpr`: garantisce che una variabile con durata statica o thread-local venga inizializzata a tempo di compilazione, prevenendo lo Static Initialization Order Fiasco, un problema che sorge quando l'ordine di inizializzazione di variabili statiche tra diverse unità di traduzione non è definito, potendo portare all'uso di variabili non inizializzate.

```
1 //file a.cpp
2 int value(){ return 50; }
3
4 int y = value(); //inizializzazione a runtime
5
6
7 //file b.cpp
8 #include <iostream>
9 extern int y;
10
11 int x = y; //la funzione value() potrebbe non essere stata
    eseguita
12
13 int main(){
14     std::cout<<"x: "<<x<<std::endl;
15     std::cout<<"y: "<<y<<std::endl;
16 }
17
18 //Compilazione: g++ -std=c++20 a.cpp b.cpp -o test
19 //Output: "x: 50" "y: 50"
20
21 //Compilazione: g++ -std=c++20 b.cpp a.cpp -o test
22 //Output: "x: 0" "y: 50" ERRORE, y utilizzato prima di
    essere inizializzato
23
24 //Soluzione: modifico a.cpp
25 constexpr int value(){ return 50; }
26
27 constexpr int y = value(); //inizializzazione a compile
    time
28 //Output: "x: 50" "y: 50"
```

2.4 Metaprogramming nel C++26

Il futuro della metaprogrammazione nel C++ è incentrato sulla reflection e su sintassi più intuitive per manipolare i tipi e le strutture del programma. La reflection consiste nella capacità di un programma di osservare i propri tipi, ottenendo informazioni sui membri di una classe, le basi di una gerarchia, gli argomenti di una funzione o i parametri di un template a tempo di compilazione. Queste informazioni, accessibili tramite meta-objects, possono essere usate per generare codice in modo dichiarativo. Un applicazione pratica di questo meccanismo è l'unpacking automatico di strutture dati, che permetterebbe di iterare sui membri di una classe o serializzarla/deserializzarla senza dover scrivere codice boilerplate manualmente per ogni membro. Combinata con `constexpr`, la reflection permetterà di scrivere codice generico efficiente (risolto a compile-time) e facile da mantenere (il codice ripetitivo è ridotto drasticamente).

Esempio: da struct a JSON

La proposta di reflection per il C++26 utilizza una sintassi particolare per accedere alle meta-informazioni. Non essendo ancora implementata in nessun compilatore, la sintassi potrebbe essere soggetta a variazioni. Segue una breve spiegazione dei costrutti che verranno utilizzati nell'esempio:

- `T^(meta-type operator)`: L'operatore `"^"` applicato ad un tipo (`T^`) restituisce il meta-tipo di `T`, ovvero un oggetto che rappresenta la dichiarazione del tipo `T`, e permette di accedere alle sue proprietà.
- `meta_type::data_members`: Il meta-tipo di una classe (qui `meta_type`) fornisce un array statico chiamato `data_members` che contiene le meta-informazioni di tutti i suoi membri dati.
- `for... (constexpr auto : ...)`: È un nuovo tipo di ciclo `for` valutato a tempo di compilazione. Itera in modo sequenziale su tutti gli elementi di una sequenza (in questo caso l'array `data_members`), istanziando il corpo del ciclo per ogni elemento.
- `membro.name`: Ogni elemento in `data_members` è un meta-oggetto che rappresenta un membro. La proprietà `".name"` fornisce il nome del membro sotto forma di stringa compile-time.
- `obj.[membro]`: Questa è la sintassi per accedere ai membri via meta-oggetti. Permette di accedere al membro dell'oggetto `obj` che è descritto dalla meta-informazione `membro`. Equivale a scrivere `obj.nome_membro` ma in modo generico, usando la meta-informazione invece di un nome fisso.

```
1 #include <string>
```

```

2 #include <type_traits>
3
4 //METODOLOGIA CORRENTE
5 struct Persona{
6     std::string nome;
7     int eta;
8 };
9
10 std::string serializza(const Persona& p){
11     return "{\"nome\": \"" + p.nome + "\", \"eta\": \" " + std::
        to_string(p.eta) + "\"}";
12 } //se aggiungo un campo alla struct, questa funzione va
        modificata
13
14 // APPROCCIO CON REFLECTION
15 template <typename T>
16 std::string serializza(const T& obj) {
17     std::string json = "{";
18     //ottiene il meta-tipo di T
19     typedef typename T^ meta_type;
20     // Itera sui membri dati di T
21     for... (constexpr auto membro : meta_type::data_members) {
22         json += "\"" + std::string(membro.name) + "\": ";
23         //usa if constexpr per distinguere il tipo del membro
24         if constexpr (std::is_integral_v<decltype(obj.[:membro:])>
25             >) {
26             json += std::to_string(obj.[:membro:]);
27         } else {
28             json += "\"" + obj.[:membro:] + "\"";
29         }
30         json += ", ";
31     }
32     if (!json.empty() && json != "{") {
33         json.pop_back(); //rimuove l'ultima virgola
34         json.pop_back();
35     }
36     json += "}";
37     return json;
38 }
39 //UTILIZZO:
40 auto json = serializza(Persona{"Sebastiano",23});

```

Nonostante la versione che utilizza reflection sia concettualmente più complessa, i vantaggi a lungo termine sono molteplici:

- Codice generico: la funzione serializza funziona per qualsiasi tipo di struttura, salvo casi particolari. L'aggiunta di nuovi campi a Persona o la creazione di nuove struct non richiede lavoro aggiuntivo per la serializzazione. Nel caso in cui i nuovi campi aggiunti a Persona abbiano tipi più complessi (std::vector,

tipi annidati), la logica del blocco `if constexpr` andrebbe estesa. Questo procedimento andrebbe però fatto solo all'interno della funzione generica, a beneficio di tutte le struct esistenti e future.

- **Manutenibilità:** Il codice boilerplate viene scritto una sola volta. Correggere bug o migliorare la formattazione del JSON richiede di modificare un'unica funzione.
- **Minore probabilità di errore:** Utilizzando l'approccio tradizionale, creare svariate copie della stessa funzione modificata per ogni struct è un processo propenso ad errori (esempio: typo nel nome di un campo JSON). Questa intera classe di errori viene eliminata utilizzando l'approccio reflection.

Capitolo 3

Gestione della Memoria

3.1 Gestione manuale delle risorse

Il C++, ereditando il modello di gestione della memoria dal C, ha affidato al programmatore la responsabilità della gestione delle risorse attraverso gli operatori `new` e `delete`. Questo approccio, nonostante offra un elevato livello di controllo sulle operazioni, è una delle maggiori fonti di bug e vulnerabilità delle applicazioni. I puntatori classici infatti, detti *raw pointers*, se gestiti incorrettamente possono portare a:

- Wild pointers: puntatori non inizializzati, il cui utilizzo causa comportamento indefinito.

```
1 int* wild_pointer; //valore imprevedibile
2 int something = *wild_pointer; //Undefined behavior
```

- Memory leak: se il programmatore dimentica di deallocare utilizzando `delete` aree di memoria allocate dinamicamente con `new`, la memoria viene persa per l'intero programma. Questo fenomeno può verificarsi anche se si verifica un'eccezione prima del `delete`, interrompendo il flusso di esecuzione.

```
1 void leak(){
2     int* pointer = new int(5);
3 } //perdo il riferimento all'area di memoria: non potrò più
    deallocarla!
```

- Dangling pointers: puntatori che puntano ad aree di memoria già liberate.

```
1 int* dangling = new int(6);
2 delete dangling; //dealloco la memoria
3 *dangling = 10; //comportamento indefinito!
```

- Doppia liberazione: liberare più volte un'area di memoria già liberata, che porta ad un comportamento indefinito (Undefined Behavior).

```
1 int* dangling = new int(6);  
2 delete dangling;  
3 //...codice...  
4 delete dangling; //comportamento indefinito!!
```

3.2 Rule of Three

La comunità C++ identificò questi problemi, proponendo la "Rule of Three" come prima soluzione imperfetta. Non è altro che una linea guida, infatti stabilisce che se una classe necessita di definire un distruttore personalizzato per gestire una risorsa, allora molto probabilmente ha bisogno anche di un costruttore di copia e di un operatore di assegnamento per copia, in modo da gestire correttamente la copia di quella risorsa. La regola è dovuta al comportamento del C++, in quanto se non vengono dichiarati i costruttori di copia e assegnamento, vengono generati automaticamente. Gli operatori generati fanno una copia superficiale (member-wise copy), disastrosa per risorse come puntatori, che richiedono copie profonde (deep copy) o gestioni particolari. Può infatti portare a situazioni in cui il distruttore di una classe libera la memoria, lasciando le classi rimanenti con un dangling pointer. Nonostante prevenisse molti errori, era fortemente limitata dalla necessità delle copie profonde (deep copy), in quanto molto costose e spesso non necessarie.

3.3 Il principio RAI

Il principio Resource Acquisition Is Initialization (RAII) è alla base della gestione delle risorse nel C++ moderno. Stabilisce che l'acquisizione di una risorsa dovrebbe avvenire durante l'inizializzazione di un oggetto, e il rilascio dovrebbe avvenire durante la sua distruzione. Questo principio sfrutta una delle caratteristiche fondamentali del C++, ovvero la distruzione deterministica degli oggetti: quelli con durata automatica (sullo stack) vengono distrutti quando escono dallo scope, mentre quelli con durata statica alla chiusura del programma. Viene garantita dunque la liberazione della risorsa anche nel caso si verifichi un'eccezione, dunque rilasciando la risorsa indipendentemente dal modo in cui si esce dallo scope. In questo modo, la responsabilità della gestione di una risorsa viene sollevata dal programmatore, rendendo la programmazione meno soggetta a errori dovuti al rilascio manuale delle risorse.

```
1 //esempio: gestione di un file con fstream  
2 #include <string>  
3 #include <fstream>
```

```
4 #include <iostream>
5
6 void gestore_file(const std::string& nome_file){
7     //acquisisco la risorsa nell'inizializzazione
8     std::ofstream file(nome_file);
9
10    //uso la risorsa
11    file << "Informazioni..." << std::endl;
12
13 } //NON è necessario chiamare file.close(). La risorsa viene
    rilasciata automaticamente con la chiamata del distruttore di
    "file", anche in caso di eccezioni
```

3.4 Move Semantics

Nonostante il principio RAI risolvesse il problema della gestione della durata delle risorse, il problema dell'inefficienza delle copie non necessarie rimaneva senza soluzione. Le move semantics, introdotte nel C++11, sono nate proprio con lo scopo di risolvere questo problema. Il concetto di base consiste nel permettere lo spostamento (move) dello stato di un oggetto, trasferendo le proprietà delle risorse interne, invece che creare una copia completa. Questo è reso possibile dalle rvalue reference, dichiarate con &&, che permettono di identificare e manipolare oggetti temporanei o, in generale, oggetti da cui è possibile spostare le risorse in quanto non verranno più utilizzati. Gli spostamenti sono effettuati utilizzando la funzione `std::move`, che di per sé non compie nessuno spostamento. Il suo ruolo è quello di eseguire un cast a rvalue reference, segnalando al compilatore che un oggetto, nonostante non sia temporaneo, può essere trattato come tale, permettendo di conseguenza lo spostamento delle risorse. Lo spostamento vero e proprio è implementato nel move constructor e nel move assignment operator della classe dell'oggetto.

```
1 #include <vector>
2 #include <utility>
3
4 std::vector<int> v1(1000000,0);
5
6 //copio il vettore (costruttore di copia), 0(n)
7 std::vector<int> v2 = v1;
8
9 //sposto il vettore (costruttore di move), 0(1)
10 std::vector<int> v3 = std::move(v1);
11
12 //dopo il move, v1 è in stato indefinito
```


3.4.1 Perfect Forwarding

Il perfect forwarding è una tecnica che risolve un problema specifico, ovvero la possibilità di scrivere funzioni template (soprattutto wrapper) che non modifichino la categoria di valore (dunque se lvalue o rvalue) degli argomenti che vengono passati, per poterli inoltrare a loro volta ad altre funzioni in maniera efficiente. Questo meccanismo è essenziale per evitare copie quando gli argomenti vengono passati attraverso strati di template. Si basa su due concetti:

- Riferimenti universali: In contesti template con deduzione del tipo, T&& non significa necessariamente "rvalue reference a T", ma diventa un riferimento universale, in grado di legarsi sia a lvalue che rvalue.
- Reference collapsing: le regole del linguaggio "collassano" i riferimenti multipli secondo la regola: un riferimento ad un riferimento diventa un riferimento semplice, tranne nel caso in cui due rvalue reference vengono combinati (T&& && collassa in T&&).

La funzione `std::forward` (con sintassi `std::forward<T>(arg)`) sfrutta queste regole: consiste in un cast condizionato che restituisce un lvalue reference o un rvalue reference a seconda della categoria originale del parametro (lvalue o rvalue). Di conseguenza è possibile scrivere funzioni che accettano riferimenti universali (dichiarati come T&&) e inoltrarli senza modificarne la categoria.

```
1 #include <iostream>
2 #include <utility>
3
4 template <typename T>
5 void funzione_generica(T&& arg) {
6     if constexpr (std::is_lvalue_reference_v<T&&>) {
7         std::cout << "Chiamato con LVALUE: " << arg << std::endl;
8     } else {
9         std::cout << "Chiamato con RVALUE: " << arg << std::endl;
10    }
11 }
12
13 //funzione di perfect forwarding
14 template<typename T>
15 void perfect_forwarding(T&& arg){
16     //inoltrato come lvalue o rvalue, a seconda di T
17     funzione_generica(std::forward<T>(arg));
18 }
19
20 int main() {
21     int x = 10;
22
23     perfect_forwarding(x); //passaggio di un lvalue
24     perfect_forwarding(std::move(x)); //passaggio di un rvalue
```

```

25
26     return 0;
27 }

```

3.5 Rule of Five

Con l'arrivo delle Move Semantics, la "Rule of Three" è diventata la "Rule of Five". Infatti ora le classi necessitavano due nuovi membri speciali, già menzionati nella sezione precedente, per gestire i casi di spostamento:

- Costruttore di spostamento: ha come scopo quello di trasferire le risorse da un oggetto temporaneo o reso spostabile (tramite `std::move`) durante la costruzione di un nuovo oggetto. Dopo essersi appropriato delle risorse, ha anche come compito quello di rimuovere eventuali puntatori e handle dall'elemento temporaneo impostandoli su uno stato vuoto, in modo da evitare che il distruttore dell'elemento rilasci le risorse che sono di proprietà del nuovo oggetto.
- Operatore di assegnamento per spostamento: ha come scopo quello di trasferire le risorse da un oggetto temporaneo ad uno già esistente. Di conseguenza, deve

- 1 Rilasciare le risorse correnti dell'oggetto corrente
- 2 Prelevare le risorse dell'oggetto temporaneo
- 3 Impostare le risorse dell'oggetto temporaneo su uno stato vuoto.

Essenzialmente, esegue le stesse operazioni del costruttore di spostamento, dopo aver liberato le risorse correnti, in quanto a differenza del costruttore, l'oggetto esiste già. Inoltre, deve gestire correttamente l'autoassegnamento (`x = std::move(x)`), nonostante sia una evenienza rara.

Entrambi i membri devono essere `noexcept`, ovvero segnalano al compilatore che non si verificheranno eccezioni al loro interno. In caso contrario molte funzioni della standard library, come ad esempio `std::vector::resize`, preferiranno utilizzare la copia, per evitare di perdere dati in caso di eccezioni, con conseguente calo di prestazioni.

```

1  #include <utility>
2
3  class Handler{
4      int* dati; //risorsa da gestire
5
6      /*altri membri, omessi per brevità*/
7
8      //Costruttore di spostamento
9      Handler(Handler&& other) noexcept : dati(other.dati){
10         //rimuove il riferimento dall'oggetto sorgente

```

```

11         other.dati = nullptr;
12     }
13
14     //Operatore di assegnamento per spostamento
15     Handler& operator=(Handler&& other) noexcept{
16         if(this != &other){ // controllo per autoassegnamento
17
18             //rilascia la risorsa corrente
19             delete[] dati;
20
21             //preleva la risorsa
22             dati = other.dati;
23
24             //rimuove il riferimento
25             other.dati = nullptr;
26         }
27         return *this;
28     }
29
30     //nota: il distruttore deve poter gestire il caso senza
31     riferimenti. In questo caso, delete[] nullptr è sicuro
32     ~Handler(){ delete[] dati; }
33 }
34
35 //Utilizzo:
36 Handler ob1; //costruttore di default
37 Handler ob2 = std::move(ob1); //chiama il move constructor, ob1
38 ora non possiede più la risorsa
39
40 Handler ob3; //costruttore di default
41 ob3 = std::move(ob2); //chiama il move assignment operator, ob2
42 ora non possiede più la risorsa

```

3.6 Smart Pointers

Gli smart pointers, inclusi nell'header `memory`, sono delle classi che gestiscono autonomamente il ciclo di vita degli oggetti allocati dinamicamente. Incarnano il principio RAII, in quanto si comportano da wrapper per i raw pointer. Possono essere utilizzati con una sintassi simile ai puntatori classici grazie al sovraccarico degli operatori `*` e `->`. Si prendono carico della gestione di una zona di memoria nel costruttore (tramite allocazione o ricevendo un puntatore già inizializzato), e ne garantiscono la deallocazione nel distruttore, eliminando il rischio di memory leak. Esistono tre tipi di smart pointers, ognuno progettato per uno scenario specifico di gestione della memoria.

3.6.1 Unique Pointers

Lo `std::unique_ptr` rappresenta il concetto di possesso esclusivo: un'unica entità possiede la risorsa, e ne garantisce la deallocazione. Non possono essere copiati, in quanto violerebbe il processo esclusivo, implementato tramite la cancellazione del costruttore di copia e operatore di assegnamento per copia tramite la keyword `deleted`; un tentativo di copia corrisponderà di conseguenza ad un errore di compilazione. L'integrazione con le move semantics è fondamentale in quanto i puntatori possono essere spostati, preservando la semantica di possesso esclusivo. Presentano un overhead praticamente nullo rispetto ai puntatori "raw", dopo le ottimizzazioni del compilatore.

```
1 #include <utility>
2 #include <memory>
3
4 //creo un unique_ptr che possiede la risorsa
5 std::unique_ptr<int> p1(new int(5));
6
7 //errore! non può essere copiato
8 //std::unique_ptr<int> p2 = p1;
9
10 //può essere spostato, trasferendo l'ownership
11 std::unique_ptr<int> p2 = std::move(p1);
12
13 //p1 ora è vuoto
14 //la deallocazione avverrà automaticamente
15
16 //dal C++14: definizione più sicura
17 auto p1 = std::make_unique<int>(5);
18
19 //C++20: estrazione del puntatore grezzo:
20 int* raw = std::to_address(p1);
21
22 //C++23: unique_ptr può essere usato in contesti constexpr
23 constexpr int test(){
24     auto p = std::make_unique<int>(6);
25     return *p;
26 }
```

Si preferisce utilizzare la funzione `std::make_unique()` per la definizione di `unique_ptr`, in quanto con la definizione classica (riga 2) si corre il rischio di memory leak se si verifica un'eccezione tra l'allocazione della memoria (tramite `new`) e l'acquisizione della memoria tramite il costruttore.

3.6.2 Shared Pointers

Se è necessario l'utilizzo di una risorsa comune, si utilizzano gli `std::shared_ptr`. Questo tipo di puntatori si occupa comunque della gestione della vita dell'ogget-

to, ma in maniera differente rispetto agli `std::unique_ptr`: essendo che l'oggetto è condiviso, questo verrà distrutto una volta che tutti i riferimenti a quell'oggetto sono stati distrutti. Di conseguenza è necessario tenere in memoria un contatore di riferimenti, che tiene traccia del numero di `std::shared_ptr` che puntano a quella risorsa. L'overhead è decisamente più elevato rispetto alla controparte `unique`, dovuto al fatto che è necessario allocare il blocco di controllo, contenente le informazioni necessarie per il reference counter, inoltre è vitale l'utilizzo di operazioni atomiche nella manipolazione del reference count, in modo da garantire consistenza in contesti multithread.

```
1 #include <memory>
2 #include <iostream>
3
4 //crea uno shared_ptr
5 std::shared_ptr<int> s1(new int(5));
6
7 //possono essere copiati -> incremento reference count
8 std::shared_ptr<int> s2 = s1; // count = 2;
9
10 //rilascio manuale della risorsa -> decrementa reference count
11 s1.reset(); //count = 1;
12
13 //si può continuare ad accedere alla zona di memoria
14 std::cout << *s2 << std::endl; //stessa sintassi di un puntatore
    normale!
15
16 //con rilascio manuale o automatico della risorsa, reference
    count = 0 e la risorsa verrà deallocata
17 s2.reset();
18
19 //Definizione più sicura
20 auto ss = std::make_shared<int>(5);
21
22 //C++20: shared_ptr supporta factory sicure per array
23 auto arr = std::make_shared<int[]>(5);
24 arr[0] = 10;
```

A differenza degli `unique_ptr`, la funzione `std::make_shared` è presente sin dal C++11, introdotta assieme agli smart pointers. Oltre al vantaggio della sicurezza, già menzionato per `std::make_unique`, questa funzione permette di creare con un'unica allocazione sia l'oggetto gestito sia il blocco di controllo, un approccio più efficiente rispetto a due allocazioni separate.

3.6.3 Weak Pointers

Gli `std::shared_ptr` non eliminano completamente la possibilità di generare memory leak. Infatti, se si genera un ciclo di riferimenti tramite due o più strutture dati che

si puntano a vicenda con degli smart pointers, all'uscita dallo scope il contatore non verrà decrementato e i distruttori non verranno chiamati, generando memory leak. Per spezzare i cicli di riferimenti, si utilizzano gli `std::weak_ptr`, che forniscono un riferimento ad un oggetto senza però incrementarne il reference count. Viene comunque tenuta traccia di quanti `weak_ptr` stanno osservando un oggetto, tramite un weak count. A differenza del reference count, questo contatore evita la deallocazione del blocco di controllo (dove sono contenuti sia il reference count che il weak count), permettendo agli `weak_ptr` di continuare ad accedervi. Questa particolarità è fondamentale per il funzionamento in quanto per poter accedere all'oggetto è necessario convertirlo temporaneamente in uno `shared_ptr` tramite il metodo `lock()`, che controlla se la risorsa è ancora allocata tramite il blocco di controllo. Quando tutti gli `weak_ptr` verranno distrutti, anche il blocco di controllo verrà deallocato.

Il Problema: ciclo di riferimenti

```
1 #include <memory>
2 #include <iostream>
3
4 class B; //dichiarazione in anticipo
5
6 class A{
7     public:
8         std::shared_ptr<B> b_ptr;
9         ~A(){ std::cout << "A distrutto" << std::endl;}
10 };
11
12 class B{
13     public:
14         std::shared_ptr<A> a_ptr;
15         ~B(){ std::cout << "B distrutto" << std::endl;}
16 };
17
18 int main(){ //creo i due oggetti
19     auto elem_a = std::make_shared<A>();
20     auto elem_b = std::make_shared<B>();
21
22     elem_a->b_ptr = elem_b;
23     elem_b->a_ptr = elem_a; //ciclo! il contatore non andrà mai a
    zero
24     return 0;
25 } //ne A ne B vengono distrutti, si verifica memory leak!
```

La Soluzione

Spezziamo il ciclo sostituendo uno `shared_ptr` con un `weak_ptr`.

```

1 //...
2 class B{ //modifico la definizione di B
3     public:
4         std::weak_ptr<A> a_weak_ptr;
5         ~B(){ std::cout << "B distrutto" << std::endl;}
6 };
7
8 int main(){
9     auto elem_a = std::make_shared<A>();
10    auto elem_b = std::make_shared<B>();
11
12    elem_a->b_ptr = elem_b;
13    elem_b->a_weak_ptr = elem_a; //il reference counter di A non
    verrà incrementato
14
15    //per accedere ad A da B tento di convertire il weak_ptr in
    shared_ptr
16    if(auto shared_a = elem_b->a_weak_ptr.lock())
17        std::cout << "Accesso eseguito!" << std::endl;
18    else
19        std::cout << "A non esiste più." << std::endl;
20
21    return 0;
22 } //tutti gli oggetti verranno distrutti correttamente

```

3.7 Nuove Proposte

- `indirect<T>`: un "vocabulary type" che agisce come smart pointer ma con semantiche diverse. Gestisce un oggetto come un `unique_ptr` copiabile.

```

1 #include <experimental/indirect>
2 #include <string>
3 #include <iostream>
4
5 std::indirect<std::string> s1 = "Hello";
6 auto s2 = s1; //copia profonda, non c'è condivisione
7 *s2 = "World";
8
9 std::cout << *s1 << std::endl; // "Hello"
10 std::cout << *s2 << std::endl; // "World"

```

- `mixed comparisons`: permette di confrontare raw pointer e smart pointer

```

1 #include <memory>    // make_unique, to_address
2 #include <iostream>
3

```

```
4 auto up = std::make_unique<int>(5);
5 int* raw = std::to_address(up);
6
7 if(up == nullptr){/*...*/} //già presente
8 if(up == raw){/*...*/} //mixed comparison
```

3.8 Scope Guards

Gli scope guards (introdotti in C++23) permettono di eseguire automaticamente del codice alla fine di uno scope, permettendo il controllo su quando deve essere eseguito. Essenzialmente sono dei piccoli wrapper che eseguono una certa funzione lambda nel distruttore, scelta a seconda dell'esito dell'operazione. Sono presenti tre varianti:

- `std::scope_exit`: esegue la funzione assegnata quando si esce dallo scope, a prescindere dal modo;
- `std::scope_success`: esegue la funzione assegnata solo se si esce dallo scope normalmente, senza eccezioni;
- `std::scope_error`: esegue la funzione solo se si esce dallo scope tramite un'eccezione.

```
1 #include <iostream>
2 #include <scope>
3 #include <stdexcept>
4
5 void operation(bool throw_err){
6     std::scope_exit{[] {std::cout << "Uscito" << std::endl;}}
7     std::scope_success{[] {std::cout << "Successo" << std::endl;}};
8     std::scope_fail{[] {std::cout << "Errore" << std::endl;}};
9
10    /*...working...*/
11    if(throw_err)
12        throw std::runtime_error("errore");
13 }
14
15 operation(false);
16 //output: "Successo" poi "Uscito"
17 try{
18     operation(true);
19 }catch(...){}
20 //output: "Errore" poi "Uscito"
```


3.9 Gestione Dinamica Efficiente con `std::hive`

La proposta di `std::hive` per lo standard C++26 è stata motivata dalla serie di limitazioni intrinseche presenti negli attuali container sequenziali della STL, in particolare in scenari che richiedono inserimenti e cancellazioni frequenti in posizioni casuali. La potenza di `std::hive` è dovuta alla propria architettura a blocchi di memoria collegati, che comporta numerosi vantaggi:

- A differenza di `std::vector`, che utilizza un array contiguo che necessita costose riallocazioni con conseguente invalidazione degli iteratori associati, `std::hive` evita riallocazioni totali, mantenendo inoltre stabili i riferimenti agli elementi esistenti.
- A differenza di `std::list`, che alloca gli elementi separatamente con conseguente frammentazione della memoria e scarsa località dei riferimenti, `std::hive` alloca la memoria in blocchi predefiniti o di dimensione variabile, riducendo l'overhead dovuto all'allocazione e migliorando l'efficienza della cache.
- Gestione efficiente della memoria tramite il riutilizzo degli slot liberati da cancellazioni.

```
1 #include <hive>
2 #include <string>
3 #include <iostream>
4
5 struct Persona{
6     std::string nome;
7     int eta;
8
9     Persona(std::string n, int e) : nome(std::move(n)), eta(e) {}
10 };
11
12 std::hive<Persona> container;
13
14 //Inserimento elementi
15 container.insert(Persona{"Bruno",26});
16 container.insert(Persona{"Marcello",34});
17
18 //è possibile definire un iteratore legato al valore inserito
19 auto seba = container.insert(Persona{"Sebastiano",23});
20 auto nora = container.insert(Persona{"Nora",18});
21
22 //Costruzione sul posto
23 container.emplace("Mario",50);
24
25 //Cancellazione sicura
26 container.erase(nora); //L'iteratore nora non è piu valido!
27
```

```
28 //Gli iteratori rimangono validi dopo inserimenti e cancellazioni
29 std::cout << seba->eta << std::endl; //Output: 23
30
31 //Inserimenti successivi riutilizzeranno gli slot liberati
32 auto ugo = container.insert(Persona{"Ugo",40});
33
34 //La memoria è divisa in blocchi, non contigua
35 std::cout << "Elementi: " << container.size();
36 std::cout << " Blocchi usati: " << container.block_capacity();
37
38 //Iterazione con iteratori espliciti:
39 for(auto it = container.begin(); it != container.end(); ++it){
40     std::cout << it->nome << " - " << it->eta << std::endl;
41 }
42
43 //Sono supportate tutte le feature moderne del C++
44 //Range-based for loop
45 for(const auto& elem : container){
46     std::cout << elem->nome << std::endl;
47 }
48
49 //Move semantics
50 std::hive<Persona> container_2 = std::move(container);
```

Capitolo 4

Programmazione Asincrona

4.1 Multithreading nel passato

Lo standard C++98/03 non prevedeva nessuna libreria per la programmazione multithreading. Di conseguenza, era necessario utilizzare API di sistema o librerie esterne, come i POSIX threads (pthread) su Unix, o librerie di terze parti come "Boost.Thread". Non esisteva infatti un modello di memoria standardizzato per la concorrenza, ovvero non c'era una specifica chiara sul come le operazioni su memoria condivisa venissero viste da thread diversi. Il comportamento dipendeva da vari fattori, come il compilatore e la piattaforma, rendendo praticamente impossibile scrivere del codice portabile. Nonostante la mancanza di supporto ufficiale, la filosofia RAII era già usata in librerie come Boost per gestire mutex e lock.

4.2 Supporto nativo del multithreading

Il C++11 è stato il punto di svolta per la programmazione concorrente, perché per la prima volta sono stati forniti strumenti ufficiali, portabili e sicuri per lavorare con i thread. Inoltre, è stato definito un modello di memoria condiviso, che chiarisce il comportamento dei thread quando interagiscono, ad esempio con l'accesso ad una variabile comune.

```
1 #include <iostream>
2 #include <thread>
3
4 void hello(){
5     std::cout << "Sono il thread" << std::endl;
6 }
7
8 int main(){
9     std::thread son(hello); //crea e avvia il thread
10    son.join(); //attende la terminazione del thread
11    return 0;
12 }
```

4.3 Strumenti del multithreading

Per poter programmare in sicurezza, è necessario garantire la mutua esclusione e l'atomicità di un'operazione quando si utilizza una zona di memoria condivisa, oppure garantire un certo ordine alle operazioni. Di conseguenza, lo standard ha definito i classici meccanismi di sincronizzazione, includendo anche alcune novità.

- **Mutex**: strumento classico per la sincronizzazione. Permettono di delimitare zone critiche di codice, che solitamente contengono la gestione di risorse condivise, che possono essere eseguite da un unico thread per volta in modo da garantire uno stato consistente delle risorse.

```
1 #include <iostream>
2 #include <vector>
3 #include <thread>
4 #include <mutex>
5
6 std::mutex m;
7
8 void work(int id){
9
10     //prendo la mutua esclusione
11     m.lock();
12
13     /*...manipolo risorse (stato inconsistente)...*/
14     std::cout << "Sono il thread " << id << std::endl;
15
16     //una volta terminate le operazioni, le risorse sono in
17     //uno stato consistente: rilascio il lock
18     m.unlock();
19 }
20
21 int main(){
22     int N = 5;
23     std::vector<std::thread> thread_vect;
24     for(int i = 0; i < 5; i++)
25         thread_vect.emplace_back(work,i); //crea e avvia N
26         //thread, inserendoli nel vettore thread_vect
27     for(auto& t : thread_vect)
28         t.join(); //attende la terminazione di ogni thread
29     return 0;
30 }
```

L'utilizzo diretto delle funzioni lock/unlock è sconsigliato, in quanto se ci si dimentica di liberare il mutex, oppure si verifica un'eccezione tra il lock e l'unlock, il mutex rimarrà bloccato. Per evitare questa eventualità, viene applicato il principio RAII anche nella gestione di questi oggetti.

```
1 #include <iostream>
2 #include <thread>
3 #include <mutex>
4
5 std::mutex m;
6
7 //LOCK GUARD
8 void work(int id){
9
10     //prendo la mutua esclusione
11     std::lock_guard<std::mutex> guard(m);
12
13     /*...manipolo risorse (stato inconsistente)...*/
14     std::cout << "Sono il thread " << id << std::endl;
15
16     //lock_guard gestisce automaticamente il rilascio del
17     mutex, che avverrà alla fine dello scope anche in caso di
18     eccezioni. NON può essere rilasciato manualmente
19 }
20
21 //UNIQUE LOCK
22 void work(int id){
23
24     std::unique_lock<std::mutex> ul(m);
25     /*...manipolo risorse (stato inconsistente)...*/
26     std::cout << "Sono il thread " << id << std::endl;
27
28     ul.unlock();
29     //...
30     if(ul.try_lock()){ //restituisce true se il mutex è stato
31         acquisito con successo
32         /*...*/
33     }
34
35     //il lock può essere gestito manualmente, ma verrà sempre
36     rilasciato alla fine dello scope
37 }
38
39 //SCOPED LOCK
40 std::mutex m,t;
41
42 void work(int id){
43     //prende entrambi i lock in modo sicuro, previene
44     deadlock
45     std::scoped_lock sl(m,t);
46     //...
47 }
```

- Condition variable: permette ad un thread di mettersi in attesa di un evento o una condizione, notificata da un altro thread.

```

1 #include <iostream>
2 #include <string>
3 #include <thread>
4 #include <mutex>
5 #include <condition_variable>
6
7 std::mutex m;
8 std::condition_variable cnd;
9 bool ready = false; //necessaria per il predicato
10 std::string message = "";
11
12 void waiting_thread(){
13     std::unique_lock<std::mutex> lock(m);
14
15     //dopo aver preso il lock, viene valutata la condizione
    nel predicato passato in ingresso alla funzione wait(). Se
    la condizione è true, il thread prosegue, altrimenti
    libera il mutex e si mette in attesa
16     cnd.wait(lock, [] { return ready; });
17     std::cout << message << std::endl; //output: "hello"
18 }
19
20 void notify_thread(){
21     std::unique_lock<std::mutex> lock(m);
22     message = "hello";
23     ready = true;
24     lock.unlock(); //rilascia il lock prima della notifica
25
26     cnd.notify_one(); //sveglia un thread in attesa
27     //cnd.notify_all(); //sveglia tutti i thread in attesa
28 }

```

- Libreria <atomic>: fornisce strumenti per gestire operazioni atomiche lock-free implementate a livello hardware per tipi semplici, come i tipi built in (int, bool, char).

```

1 #include <atomic>
2
3 std::atomic<int> at(0); //crea una variabile atomica di tipo
    int
4
5 at.store(5); //scrittura
6 int reader = at.load(); //lettura
7
8 int old = at.exchange(10) //scrive un nuovo valore e
    restituisce il vecchio
9
10 //tipi atomici predefiniti (alias per std::atomic<T>)
11 std::atomic_int a1;

```

```
12 std::atomic_bool a2;  
13 std::atomic_char a3;
```

- Future e promise: un oggetto `std::future<T>` rappresenta un valore che sarà disponibile in futuro. È di sola lettura, e serve per recuperare il risultato di un'operazione eseguita da un altro thread. Il valore viene passato attraverso l'oggetto `std::promise<T>` associato;

```
1  #include <iostream>  
2  #include <string>  
3  #include <thread>  
4  #include <future>  
5  
6  void receiving(std::future<std::string>& fut){  
7      std::string message = fut.get(); //bloccante, attende il  
      valore  
8      std::cout << message << std::endl; //output: "hello"  
9  
10     //dopo aver chiamato get(), il future non è più valido  
11     //message = fut.get(); //ERRORE! comportamento indefinito  
12 }  
13  
14 void producing(std::promise<std::string> prom){  
15     prom.set_value("message"); //imposta il valore per il  
    receiver  
16 }  
17  
18 int main(){  
19     std::promise<std::string> prom;  
20     std::future<std::string> fut = prom.get_future(); //  
    associa la promise al future  
21  
22     std::thread wt(receiving, std::ref(fut)); //avvia il  
    receiver passandogli il future  
23     std::thread nt(producing, std::move(prom)); //attiva il  
    producer passandogli la promise  
24  
25     wt.join();  
26     nt.join();  
27     return 0;  
28 }
```

- `std::async`: è una funzione di alto livello che permette di eseguire una funzione in modo asincrono restituendo un `std::future` per accedere al risultato. Permette di semplificare l'esecuzione di task asincroni senza dover gestire manualmente i thread.

```
1 #include <future>
2 #include <iostream>
3 #include <string>
4
5 std::string elabora_mess() {
6     return "hello";
7 }
8
9 int main() {
10     auto fut = std::async(std::launch::async, elabora_mess);
11     std::cout << fut.get() << std::endl; // Output: hello
12     return 0;
13 }
```

4.4 Parallel STL and std::execution

La Standard Template Library (STL) di C++ ha algoritmi paralleli integrati a partire dallo standard C++17.

Algoritmi Paralleli di C++17

Gli algoritmi STL, come `std::for_each`, `std::sort`, `std::transform` e `std::reduce`, sono stati estesi per accettare le politiche di esecuzione parallela (`std::execution::par`, `std::execution::par_unseq` e `std::execution::seq`).

L'algoritmo parallelo viene richiamato semplicemente aggiungendo una politica di esecuzione come primo argomento.

```
1 #include <vector>
2 #include <algorithm>
3 #include <numeric>
4 #include <execution>
5
6 //... riempi my_data
7 std::vector<int> my_data;
8
9 //Ordina il vettore in parallelo
10 std::sort(std::execution::par, my_data.begin(), my_data.end());
11
12 //Esegue una trasformazione in parallelo
13 std::transform(std::execution::par,
14               my_data.begin(), my_data.end(),
15               my_data.begin(),
16               [](int x) { return x * 2; });
17
18 //Riduzione parallela per calcolare la somma
```



```
19 int sum = std::reduce(std::execution::par,  
20                       my_data.begin(), my_data.end());
```

L'implementazione specifica del backend di parallelizzazione non fa parte dello standard C++, ma è gestita dal compilatore. I backend open-source più comuni sono:

- GNU GCC: La sua libreria libstdc++ usa Intel Threading Building Blocks (TBB) come backend per l'implementazione parallela della STL.
- Clang/LLVM: Clang spesso utilizza il Parallel STL del Toolkit Intel oneAPI o un'implementazione simile basata su TBB.

Quindi, non è necessario cercare una "STL parallela open-source" separata. È sufficiente assicurarsi di utilizzare un compilatore aggiornato che sfrutti un backend open-source ad alte prestazioni come TBB per l'esecuzione parallela.

4.4.1 JThread

Con il C++20 è stata introdotta la classe `std::jthread`, che rappresenta una versione migliorata di `std::thread`. Ha come scopo quello di semplificare la gestione dei thread, evitando errori comuni come la mancata chiamata a `join()`, che può provocare `std::terminate`. Infatti a differenza degli oggetti `std::thread`, un `jthread` tramite il distruttore provvede a chiamare `join()`, unendosi automaticamente al termine della sua esecuzione.

```
1 #include <iostream>  
2 #include <thread>  
3  
4 void hello(){  
5     std::cout << "Sono il jthread" << std::endl;  
6 }  
7  
8 int main(){  
9     std::jthread son(hello); //crea e avvia il thread  
10    //non è necessario chiamare son.join(): verrà fatto  
    automaticamente  
11    return 0;  
12 }
```

Stop Cooperativo

Oltre a semplificare la gestione dei thread, `std::jthread` ha introdotto un meccanismo che permette di richiedere l'interruzione di un thread in modo cooperativo. Per questo scopo sono stati introdotti due tipi:

- `std::stop_token`: è un oggetto che viene passato al thread. Durante l'esecuzione, il thread può verificare periodicamente se è stata richiesta l'interruzione tramite il metodo `stop_requested()`. Supporta anche la registrazione di callback, ovvero funzioni che vengono eseguite automaticamente quando viene richiesta l'interruzione.

```

1 #include <iostream>
2 #include <thread>
3 #include <stop_token>
4
5 void work(std::stop_token tk){
6
7     std::stop_callback callback(tk,[](){ std::cout << "
8     Richiesta Ricevuta" << std::endl;});
9
10    while(!tk.stop_requested()){
11        /*...working...*/
12    }
13    std::cout << "Interruzione..." << std::endl;
14 }
15
16 int main(){
17     std::jthread j(work); //il jthread passa automaticamente
18     uno stop_token
19     /*...working...*/
20     j.request_stop(); //segnala al thread di terminare
21
22     //il thread si joina automaticamente
23     return 0;
24 }
```

Il metodo `stop_requested()` NON forza l'interruzione immediata del thread, ma è quest'ultimo che esce volontariamente quando rileva la richiesta.

- `std::stop_source`: ogni thread ha uno `stop_source` interno di default. È possibile creare uno `stop_source` esterno in modo da controllare la stop request da più punti del programma, o anche passarla a più thread. In questo modo quando verrà chiamato `request_stop()` tutti gli `stop_token` associati verranno notificati, e di conseguenza tutti i thread interessati potranno rilevarlo tramite `stop_requested()`.

```

1 #include <iostream>
2 #include <vector>
3 #include <thread>
4 #include <stop_token>
5
6 void worker(std::stop_token st) {
7     while (!st.stop_requested()) {
```

```

8         /*...Working...*/
9     }
10    std::cout << "In uscita..." << std::endl;
11 }
12
13 int main() {
14     std::stop_source src; //stop source condivisa
15     std::stop_token tok = src.get_token(); // genera il
    token che verrà passato ai thread
16
17     std::vector<std::jthread> threads;
18
19     //crea 3 thread con un token condiviso
20     for (int i = 0; i < 3; ++i) {
21         threads.emplace_back(worker, tok);
22     }
23     /*...Working...*/
24     //richiede l'interruzione di TUTTI i thread
25     src.request_stop();
26
27     //i jthread si joinano automaticamente
28     return 0;
29 }

```

4.5 Le Coroutine

Le coroutine rappresentano il cambiamento più radicale nella programmazione concorrente dal C++11. Introdotte nel C++20, utilizzano un modello asincrono stackless, infatti a differenza dei thread (che hanno stack propri) condividono lo stack del chiamante, e lo stato viene salvato solamente tramite allocazione nell'heap. Un'altra differenza significativa consiste nel fatto che le coroutine sono di tipo cooperative: decidono loro quando cedere il controllo, il che le rende più efficienti in certi casi d'uso, come ad esempio scenari dove è richiesta concorrenza ad alto throughput senza l'overhead dei thread. Le coroutine utilizzano tre parole chiave principali:

- `co_return`: Permette di restituire un valore finale al chiamante e chiudere la coroutine.
- `co_yield`: Permettere di restituire un valore intermedio al chiamante e sospendere l'esecuzione, che potrà essere ripresa in futuro. utile per generatori o pipeline di dati.
- `co_await`: Permette di sospendere la coroutine fino al completamento di un'operazione asincrona senza bloccare il thread.

Per funzionare, le coroutine necessitano tre elementi fondamentali:

- Promise: contiene lo stato della coroutine, e si occupa di memorizzare i valori prodotti da `co_yield` e `co_return`.
- Handle: puntatore al frame della coroutine, permette di riprendere l'esecuzione o distruggerla.
- Frame: struttura allocata sull'heap, contiene le variabili locali e la promise.

Il compilatore genera automaticamente gran parte della logica interna (sospensione, salvataggio dello stato, gestione delle eccezioni), ma è compito del programmatore definire il tipo di ritorno della coroutine e la relativa promise. Questi tipi forniscono un'interfaccia per accedere ai valori prodotti e per controllare l'esecuzione della coroutine.

```

1 #include <coroutine>
2 #include <optional>
3 #include <iostream>
4
5 template <typename T>
6 struct task {
7     struct promise_type; //dichiarazione della promise
8     using handle_type = std::coroutine_handle<promise_type>;
9
10    handle_type coro; //coroutine handle
11
12    task(handle_type h) : coro(h) {}
13    task(const task&) = delete; //task non è copiabile
14    task(task&& other) noexcept : coro(other.coro) { other.coro =
        nullptr; } //move constructor
15    ~task() { if (coro) coro.destroy(); }
16
17    //metodo per riprendere la coroutine e ottenere l'ultimo
    valore restituito.
18    std::optional<T> resume() { //optional necessario per casi
        senza return
19        if (!coro.done()) { //se la coroutine non è terminata,
            prosegue l'esecuzione
20            coro.resume(); //l'handler fa ripartire la coroutine
            fino al prossimo punto di sospensione
21        }
22        if (coro.done()) //se è terminata, restituisce il valore
            restituito con co_return
23            return coro.promise().return_value_storage;
24        return coro.promise().current_value; //altrimenti il
            valore restituito con co_yield
25    }
26
27    struct promise_type {
28        std::optional<T> current_value; //ultimo valore da
            co_yield

```

```

29         std::optional<T> return_value_storage; //valore finale da
           co_return
30
31         //metodo obbligatorio
32         task get_return_object() {
33             return task{handle_type::from_promise(*this)};
34         }
35         std::suspend_always initial_suspend() { return {}; } //
           viene sospesa PRIMA di entrare nel corpo della coroutine,
           partendo solamente quando viene chiamato resume()
36         std::suspend_always final_suspend() noexcept { return {};
           } //definisce cosa fare al termine della coroutine, ritorna
           suspend_always per conservare il frame fino a destroy()
37         void unhandled_exception() { std::terminate(); } //
           gestore eccezioni
38
39         // supporto a co_return
40         void return_value(T v) { return_value_storage = std::move
           (v); }
41
42         // supporto a co_yield
43         std::suspend_always yield_value(T v) {
44             current_value = std::move(v);
45             return {};
46         }
47     };
48 };
49
50 //esempio di utilizzo
51 task<int> cl_square(int x) {
52     co_return x * x; //restituisce il quadrato
53 }
54
55 int main() {
56     auto t = cl_square(10); //la coroutine non viene eseguita
           subito, grazie a initial_suspend()
57     if (auto val = t.resume()) { //con resume(), la coroutine
           viene eseguita
58         //il risultato ottenuto è un std::optional<int>, per
           accedere al valore è necessario utilizzare "*"
59         std::cout << "Risultato: " << *val << "\n"; // Output:
           100
60     }
61 }

```

Generatori

I generatori sono un modo elegante di utilizzare le coroutine. Permettono di generare sequenze potenzialmente infinite di valori senza doverli calcolare tutti subito.

```

1 #include <coroutine>
2 #include <iostream>
3
4 //a ogni chiamata ottengo il valore successivo della sequenza
5 task<int> fibonacci(){
6     int previous = 1;
7     int next = 0;
8     while(true){
9         int aux = previous + next;
10        previous = next;
11        next = aux;
12        co_yield aux; //ritorna un valore e si sospende
13    }
14 }
15
16 //utilizzo:
17 int main() {
18     //la coroutine va definita fuori dai cicli, altrimenti verrà
    chiamato il distruttore di task a ogni iterazione
19     auto t = fibonacci();
20     while(1){
21         if (auto val = t.resume()) {
22             std::cout << "Risultato: " << *val << "\n";
23             // Output: 1,1,2...
24             if(*val > 1000)
25                 return 0;
26         }
27     }
28 }

```

Esempio con co_await

```

1 #include <coroutine>
2 #include <iostream>
3
4 //definiamo un semplice oggetto awaitable da utilizzare all'
    interno della coroutine. Non aspetta eventi reali ma simula la
    sospensione
5 struct simple_awaitable {
6     bool await_ready() { return false; } // mai pronto subito
7
8     //funzione chiamata alla sospensione della coroutine
9     void await_suspend(std::coroutine_handle<>) {
10        std::cout << "Sospeso\n";
11    }
12    //funzione chiamata quando la routine riprende l'esecuzione
13    int await_resume() {
14        std::cout << "Ripreso!\n";
15        return 100; //valore restituito dal co_await

```

```
16     }
17 };
18
19 task<int> async_example() {
20     int x = co_await simple_awaitable{}; //in attesa finché
        awaitable non "termina"
21     co_return x * 2;
22 }
23
24 int main() {
25     auto t = async_example();
26     t.resume(); //esegue la coroutine fino a co_await
27     //Output: Sospeso
28     if (auto val = t.resume()) { // Output: Ripreso!
29         std::cout << "Risultato: " << *val << "\n"; //Output: 200
30     }
31 }
```

Capitolo 5

Conclusioni

Questa tesina ha messo in evidenza l'evoluzione del C++ verso un linguaggio efficiente che permette un controllo a basso livello, e allo stesso tempo integra astrazioni di alto livello per gestione di risorse, la concorrenza e programmazione a tempo di compilazione.

Si è partiti da soluzioni manuali propense all'errore per poi passare a costrutti più avanzati, automatizzando alcune operazioni e riducendo il boilerplate, semplificando la scrittura di codice e migliorandone la qualità.

Il C++26 rappresenta il prossimo passo nell'evoluzione del linguaggio, grazie a proposte come la reflection e `std::hive` che renderanno il linguaggio più espressivo e potente, senza rinunciare alle prestazioni.

5.1 Sviluppi futuri

La tesina può essere arricchita tramite l'analisi di librerie e costrutti aggiuntivi che non sono stati affrontati, ma di uguale impatto e importanza:

- **Funzioni Lambda:** si tratta di funzioni senza nome (anonime) e definiti in linea. Permettono di scrivere una piccola funzione, senza un nome, direttamente dove serve, rendendo il codice più conciso e leggibile. Una lambda è un'entità richiamabile con un tipo unico e senza nome.
- **Lambda Generiche:** Funzioni Lambda che permettono di usare 'auto' come tipo di parametro (ossia un tipo non dichiarato ma dedotto automaticamente dal compilatore). Il compilatore genera quindi una funzione simile a un template, rendendo la lambda in grado di lavorare con qualsiasi tipo che supporti le operazioni nel suo corpo. È uno strumento potente per scrivere codice generico riutilizzabile in modo conciso.
- **Concepts:** Consentono di aggiungere dei vincoli ai parametri delle funzioni lambda generiche, Questo migliora la sicurezza dei tipi e fornisce messaggi di

errore del compilatore molto più chiari rispetto ai template tradizionali. Ad esempio, con i concepts ci si può assicurare che una lambda generica accetti solo tipi aritmetici.

- **Modules:** un sistema di importazione più efficiente, riduce i tempi di compilazione e elimina i problemi di inclusione multipla. Prendono ispirazioni a linguaggi come il Python. A regime, se verranno implementati fino in fondo, serviranno come rimpiazzo alla soluzione classica `#include "library"` e consentiranno di importare solamente alcune parti di una libreria, nonché di specificare quale versione si vuole utilizzare.
- **Filesystem:** libreria portabile che fornisce funzioni che permettono di interagire con i file del dispositivo.
- **Ranges:** funzioni che permettono di operare su sequenze di elementi in modo dichiarativo, con lazy evaluation. I ranges sono un concetto che rappresenta una sequenza di elementi. Non è un tipo di dato specifico come un `std::vector` o un `std::list`, ma piuttosto un'astrazione che unifica il modo in cui gestiamo le sequenze. In sostanza, un range è qualsiasi oggetto che puoi iterare, cioè che ha un inizio e una fine, proprio come un contenitore della libreria standard. I range sono stati introdotti in modo formale con la libreria C++20 Ranges per semplificare e rendere più leggibili le operazioni sulle sequenze di dati. Consentono la componibilità (pipelining), tramite l'operatore `|` e un approccio lazy (ossia le operazioni sui range vengono eseguite solo quando il risultato è effettivamente richiesto).
- **Sender/Receiver:** modello asincrono per operazioni concorrenti.
- **Algebra Lineare:** una libreria che fornisce operazioni matriciali e vettoriali standardizzate.
- **Contracts:** permette di definire precondizioni, postcondizioni e asserzioni a livello di linguaggio.

Inoltre possono essere analizzati più in profondità i costrutti già affrontati, tramite confronto diretto tra i costrutti moderni e tradizionali, ad esempio:

- Confronto della performance runtime tra smart pointer e raw pointer.
- Confronto tra coroutine e thread tradizionali in scenari I/O-bound e CPU-bound.
- Analisi dell'impatto sul tempo di compilazione di template ricorsivi vs funzioni `constexpr`.
- Analisi dei vantaggi di `if constexpr` rispetto alle tecniche SFINAE tradizionali.

Bibliografia

- [1] C++ reference. <https://en.cppreference.com/>.
- [2] ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee. <https://www.open-std.org/jtc1/sc22/wg21/>.
- [3] Coroutines Technical Specification for C++20 (P0912R5 and successors). <https://wg21.link/p0912>.
- [4] Reflection TS Proposal for C++26 (P1240R2 and successors). <https://wg21.link/p1240>.
- [5] std::hive Proposal for C++26. <https://wg21.link/p2549>.